# Demo Plan: Cold-Chain Alerting Engine

Nimbus Freight case study · 22CS02006 · Live: https://cold-chain-alerts-78623167451.asia-south1.run.app · Code: github.com/PRANAVTANGUTURU123/22CS02006_HCLTECH

## 1. The session at a glance

| Part | Time | What you show |
|---|---|---|
| Problem and assumptions | 3 min | Priya's problem in one line, what the seed data shows, the key assumptions (2–8 °C, 20 min, 10 min, gaps) |
| Design walkthrough | 4 min | The pipeline and the per-truck state machine; why sorting; why 20 minutes |
| Live demo (one tricky input) | 10 min | CLI report → "what if 25 min" → Discord → dashboard → own range → live new truck → Ask box (MCP) |
| Limits and next steps | 2 min | What is not built, what you would do with more time |
| Questions ("what if...") | rest | Section 7 has short answers |

**Before the session (checklist)**

- Terminal 1 in the project folder: `.\.venv\Scripts\python -m uvicorn backend.main:app --port 8000` (leave it running).
- Browser tabs: **http://127.0.0.1:8000** (dashboard), **Discord #ops-alerts**, the **live Cloud Run URL**, **GitHub**.
- Terminal 2 ready for the commands below. `.env` has the webhook URL, Groq key and admin key.
- Start clean: `Invoke-RestMethod -Method Post -Uri http://127.0.0.1:8000/api/reset -Headers @{ "X-API-Key" = "<admin key>" }`
- Close other apps and notifications; share the whole screen.

## 2. Problem and assumptions (3 min)

> **Say:** "Nimbus gets an alert for every warm reading, so ops ignore them and miss real failures. They want one alert per real, sustained problem on a truck: no alerts for blips, no repeats for the same problem, and correct even when data is missing or out of order."

| Truck | What the data shows | Result |
|---|---|---|
| T-01 | Above 8 °C 08:10–08:40, peak 9.6 °C | Alert at 08:30, closed 08:50 |
| T-02 | One reading of 8.9 °C | No alert (blip) |
| T-03 | Climbs to 7.0 °C, never above 8 | No alert |
| T-04 | Above 8 °C 08:25–08:45, **08:35 reading missing** | Alert at 08:45 (starter code missed it) |
| T-05 | Same as T-01, **shuffled in the file** | Alert at 08:30 after sorting (starter code missed it) |

**Key assumptions** (all in one config file, reasons in `docs/ASSUMPTIONS.md`):

- **2–8 °C** default range (refrigerated medicine, the strictest cargo). Each truck can have its own range; a new truck starts empty and uses the default.
- **Alert after 20 min** out of range, measured from sensor timestamps. Any limit from 10 to 20 min is correct on this data; 20 is the most patient.
- **Back to normal after 10 min** in range, seen by 2 readings. One dip does not reset or close anything.
- **Gaps up to 15 min** are bridged and reported. Longer = "sensor silent" notice. We never invent a temperature.
- **Critical** at 60 min or ≥ 11 °C. Between alert and Critical, an update only if the temperature is **not improving** (checked every 10 min).
- Humidity is shown, not alerted on: it moves with temperature (correlation 0.89).

## 3. Design walkthrough (4 min)

```
readings → Loader (validate, flag impossible temps, drop duplicates, SORT by sensor time)
         → State machine per truck:  OK → SUSPECT → ALERT → RECOVERING → OK
         → alerts + notifications (fixed ids, so each is sent once)
         → CLI report · Discord webhook · AI wording · Dashboard · MCP server
```

> **Say:** "The rules decide, the AI only explains. The engine is pure Python with no I/O, so every rule is unit-tested. Sorting matters because '20 minutes in a row' only means something in time order: T-05 shows what happens without it."

## 4. Live demo, step by step (10 min)

### Step 1: The answer on the seed data (CLI)

```
.\.venv\Scripts\python -m backend.cli
```

> **Point at:** 3 alerts from 21 warm readings; the reason line for each; "NO ALERT" explains T-02 (one reading) and T-03 (never out of range); T-04 "Includes missing data 08:30–08:40".

### Step 2: The tricky input: shuffled file and a missing reading

T-05 is shuffled in the file and T-04 has a gap. Show they are handled, then change the limit:

```
.\.venv\Scripts\python -m backend.cli --alert-after-min 25
```

> **Say:** "With 25 minutes, T-04 disappears: it was out of range for exactly 20 minutes. That's why I chose 20. It's one setting, not code." (Starter code finds only T-01; a test proves it.)

### Step 3: Alerts reach ops (Discord + AI)

```
.\.venv\Scripts\python -m backend.cli --ai --send
```

> **Show Discord:** one message per problem, AI-worded with a likely cause, and the exact facts underneath. "If the AI invents a number or is slow, the plain message goes out instead."

### Step 4: The ops dashboard

Open **http://127.0.0.1:8000** (or the live Cloud Run URL).

- Red banner: T-04 open (recovering). Fleet table with state badges.
- Click **T-04**: chart with the green safe band, the red alert period and the data gap.
- Alert cards: the rule-based reason plus the labelled AI summary.

### Step 5: Each truck's own range (no code change)

In **Truck ranges**: every truck was added automatically with an empty range (default 2–8 °C). Click **T-03**, set min 0, max 5, enter the admin key, **Save range**.

> **Say:** "T-03 at 6–7 °C is fine for medicine but too warm for chilled food: it alerts immediately. A new truck is never unprotected: an empty range means the strictest default." Then click **Use default**.

### Step 6: Live: a new truck has a problem

```
$readings = 30,35,40,45,50 | ForEach-Object { @{ truck_id = "T-09"; timestamp = "2026-07-
20T08:$($_):00"; temperature_c = 9.4; humidity_pct = 60 } }
$body = @{ readings = @($readings) } | ConvertTo-Json -Depth 4
Invoke-RestMethod -Method Post -Uri http://127.0.0.1:8000/api/readings -Headers @{ "X-API-Key" = "
<admin key>" } -ContentType "application/json" -Body $body
```

> **Show:** T-09 (never seen before) is registered, alerts on the dashboard and arrives in Discord. **Run it again:** nothing new is sent. "Each alert is sent once, and that's stored in the database, so a restart never resends." (The first upload also delivers the 3 seed alerts, which the server hadn't sent yet.)

### Step 7: Ask about the fleet (MCP)

In the **Ask about the fleet** box: "*Which trucks are in trouble right now?*", then "*What happened to T-04 in the last 30 minutes?*"

> **Say:** "Our backend is an MCP client. Groq gets the list of our MCP tools, picks one, and it runs through our MCP server on live data. The tools used are shown under the answer. They're read-only, and every number in the answer is checked against the tool results."

> Use the **local** server for this step: the Ask box has a known packaging issue on Cloud Run (see section 5). Backup: `npx @modelcontextprotocol/inspector .venv\Scripts\python -m backend.mcp_server` → click get_fleet_status.

### Step 8 (if asked): proof it works

```
.\.venv\Scripts\python -m pytest
```

221 tests: 212 Python unit + integration (97% coverage), 7 dashboard tests, 2 end-to-end browser tests. They caught 5 real bugs, including the HTTP library logging the secret webhook URL. Details: docs/TESTING.md.

## 5. Limits and next steps (2 min)

| Limit today | Next step |
|---|---|
| Batch / upload, not streaming | Streaming ingestion (Pub/Sub) with late-data handling (designed in SOLUTION_DESIGN §4) |
| Readings kept in memory in the API (sent ids and truck ranges are in the database) | Store readings in PostgreSQL; allow more than one instance |
| Ranges set per truck by ops | Range per shipment from the dispatch system; optional humidity limit |
| Impossible readings flagged but not notified | "Sensor fault" status with one notification |
| One shared admin key; no logins | Single sign-on in front of Cloud Run; per-user audit |
| Ask box on Cloud Run: the MCP helper process can't see the installed packages | Pass the full environment to the MCP subprocess (one-line fix) and redeploy |
| Tests run on Python 3.12; the cloud builds with 3.13 | Add 3.13 to the test run |

## 6. APIs (for reference)

| Endpoint | Purpose | Key? |
|---|---|---|
| `GET /health` | Liveness check | No |
| `GET /api/report` | Everything: trucks, alerts, notifications with AI summaries, data quality | No |
| `GET /api/trucks`, `GET /api/trucks/{id}` | Fleet states; one truck's readings and alerts | No |
| `GET /api/alerts?status=open\|recovering\|closed` | Alerts with reasons | No |
| `GET /api/notifications` | Notifications, AI summary, whether sent | No |
| `GET /api/ranges` | Every registered truck, its own range (or empty) and when it was added | No |
| `POST /api/ask` | Question answered by Groq via our MCP tools (20/min) | No |
| `POST /api/readings` | Upload readings (max 5,000 / 2 MB); optional ranges for new trucks | Yes |
| `PUT /api/trucks/{id}/range` | Set or clear a truck's own range | Yes |
| `POST /api/reset` | Reload the seed data (demo) | Yes |
| `/docs` | Interactive API documentation (FastAPI) | No |

**MCP tools** (read-only): get_fleet_status, get_open_alerts, get_all_alerts, get_truck_history(truck_id, minutes 1–1440), get_alert(alert_id).

**Tech stack:** Python (FastAPI, Pydantic, SQLAlchemy) on PostgreSQL; React + TypeScript + Vite; Groq `openai/gpt-oss-20b` with Gemini fallback; official MCP SDK; Discord webhook; Google Cloud Run, Cloud SQL, Secret Manager; pytest, vitest, Playwright, ruff, pip-audit.

## 7. Likely "what if..." questions

| Question | Short answer |
|---|---|
| Why 20 minutes? | On this data any limit from 10 to 20 is correct: 5 alerts on T-04's blip, 25 misses T-04. 20 is the most patient correct value, ignores door openings, and still has 3 real readings if one is missing. It's one config value. |
| What if data arrives late or out of order? | We sort by sensor time before any rule runs (T-05). Live: a late reading is put in its place and that truck re-evaluated; over 60 min late is logged but doesn't change alerts; sent alerts are marked "revised", never deleted. |
| What if the sensor goes silent? | Not treated as hot (that would bring back the noise and fake data). "Sensor silent" after 15 min, one notification at 30 min, or at 15 min as Critical if the truck was already warm. |

| | |
|---|---|
| What if a truck carries frozen food? | Set that truck's range (−25 to −18 °C) in the dashboard; no code change. Shown in step 5. |
| How do you avoid repeat alerts? | One alert per episode with a fixed id stored in the database; updates only if not improving; at most one Critical. |
| What if the AI invents something? | The AI never decides. Every number/time it writes is checked against the facts; any mismatch, error or 5 s timeout sends the plain message. |
| What if Discord is down? | 5 s timeout, 3 retries, logged; not marked sent, so it's retried next run. The alert is always on the dashboard. |
| Why not ML or a rolling average? | 54 readings can't train a model and ops can't see why it fired. A rolling average hides spikes and still flaps. The state machine is explainable and testable. |
| How is it secured? | Secrets in Secret Manager / .env only; writes need an admin key (constant-time compare); input validation and size limits; HTTPS-only webhook; tests found and fixed the webhook URL leaking into logs and the DB password showing in settings output. |
| Why MCP? | The fleet data is exposed once as read-only tools; any MCP-capable AI (Claude, or Groq in our dashboard) answers from live data instead of guessing, and can't change anything. |
| How would it scale? | Engine is per truck and stateless per evaluation; move readings to the database, ingest via Pub/Sub, run several Cloud Run instances. |

## 8. If something breaks during the demo

- **Server down:** restart Terminal 1; or switch to the live Cloud Run URL.
- **Discord silent:** run `python -m backend.cli --send` without a webhook to show the console output; the dashboard still shows every alert.
- **Groq down:** say so: alerts still go out with the plain message (that's the design). Ask box shows "AI service unavailable".
- **Data in a strange state:** run the reset command from the checklist.